Introduction to Divide and Conquer Algorithm

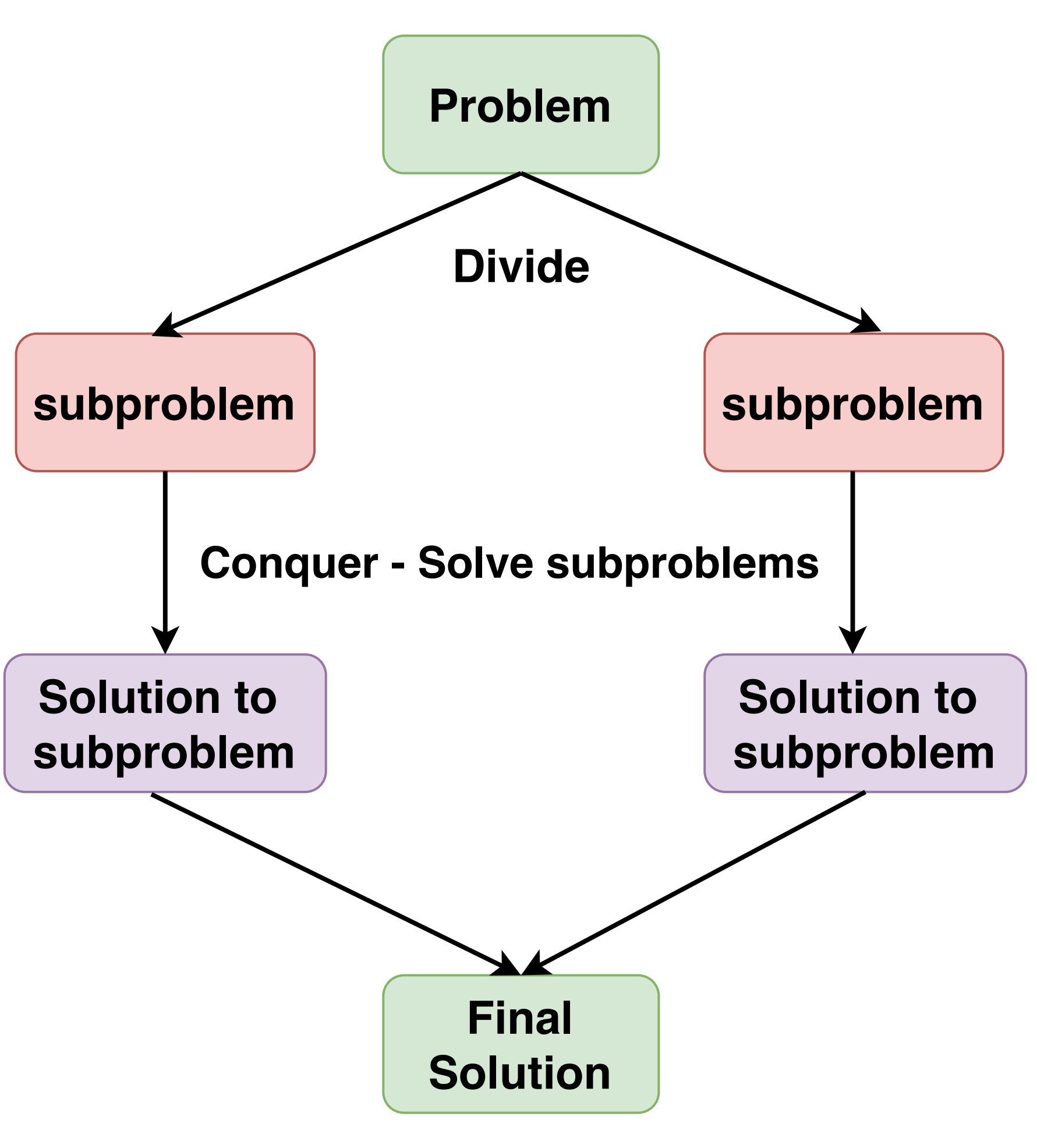
Imagine you have a huge puzzle to solve. It looks daunting at first, right? But what if you break it down into smaller pieces, solve each piece, and then put them all together? Suddenly, it seems much more manageable. This is exactly what "Divide and Conquer" does in the world of algorithms.

Divide and Conquer is a clever way to solve big problems by breaking them down into smaller, easier ones. This method is like a magic formula in computer programming, making tough tasks simpler and faster to solve. It's used in many common programs, especially for sorting and searching data.

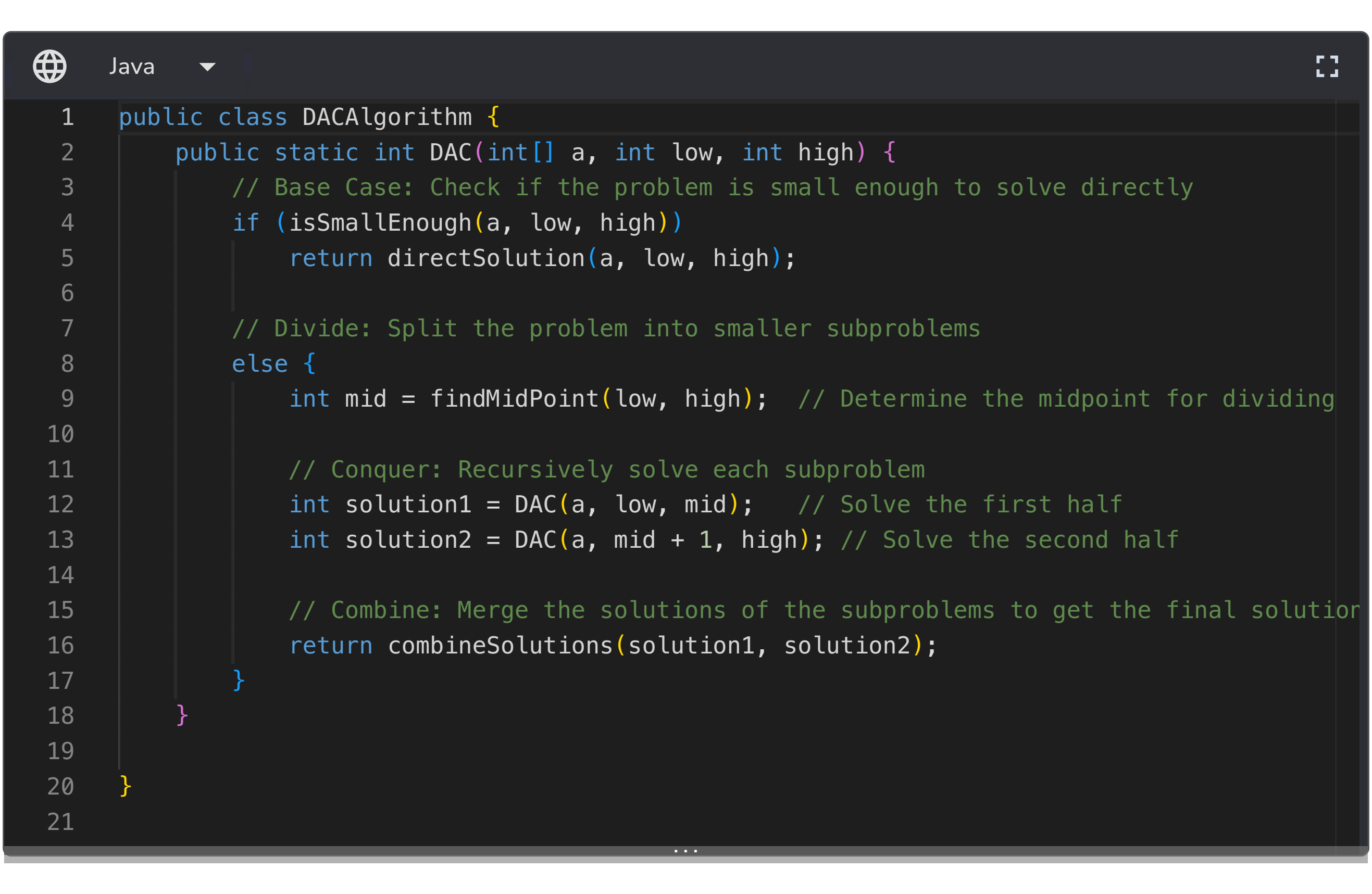
How does the Divide and Conquer Algorithm Work?

Divide and conquer algorithms work in three distinct steps:

1. **Divide:** The first step is to break the problem into smaller subproblems. This division continues until the subproblems become simple enough to be solved directly.
2. **Conquer:** Each subproblem is then solved independently. This step often involves recursive algorithmic techniques, where the function calls itself with smaller inputs.
3. **Combine:** Finally, the solutions to the subproblems are combined to form a solution to the original problem.



Here is the pseudo code for the Divide and Conquer algorithm.



```java
public class DACAlgorithm {
    public static int DAC(int[] a, int low, int high) {
        // Base Case: Check if the problem is small enough to solve directly
        if (isSmallEnough(a, low, high))
            return directSolution(a, low, high);

        // Divide: Split the problem into smaller subproblems
        else {
            int mid = findMidPoint(low, high);  // Determine the midpoint for dividing

            // Conquer: Recursively solve each subproblem
            int solution1 = DAC(a, low, mid);   // Solve the first half
            int solution2 = DAC(a, mid + 1, high); // Solve the second half

            // Combine: Merge the solutions of the subproblems to get the final solutior
            return combineSolutions(solution1, solution2);
        }
    }

}
```

Let's understand the Divide and conquer algorithm using the merge sort.

Merge Sort

Merge Sort is a divide-and-conquer algorithm that sorts an array or list by repeatedly dividing it into smaller sub-arrays or sub-lists until each small part consists of a single element or is empty. These small parts are then merged back together in a manner that results in a sorted array. The key process is the merging of two sorted sequences, which is done by comparing their elements one by one and placing them into a new array in the correct order. This method is efficient and ensures stable sorting, meaning it maintains the relative order of equal elements.

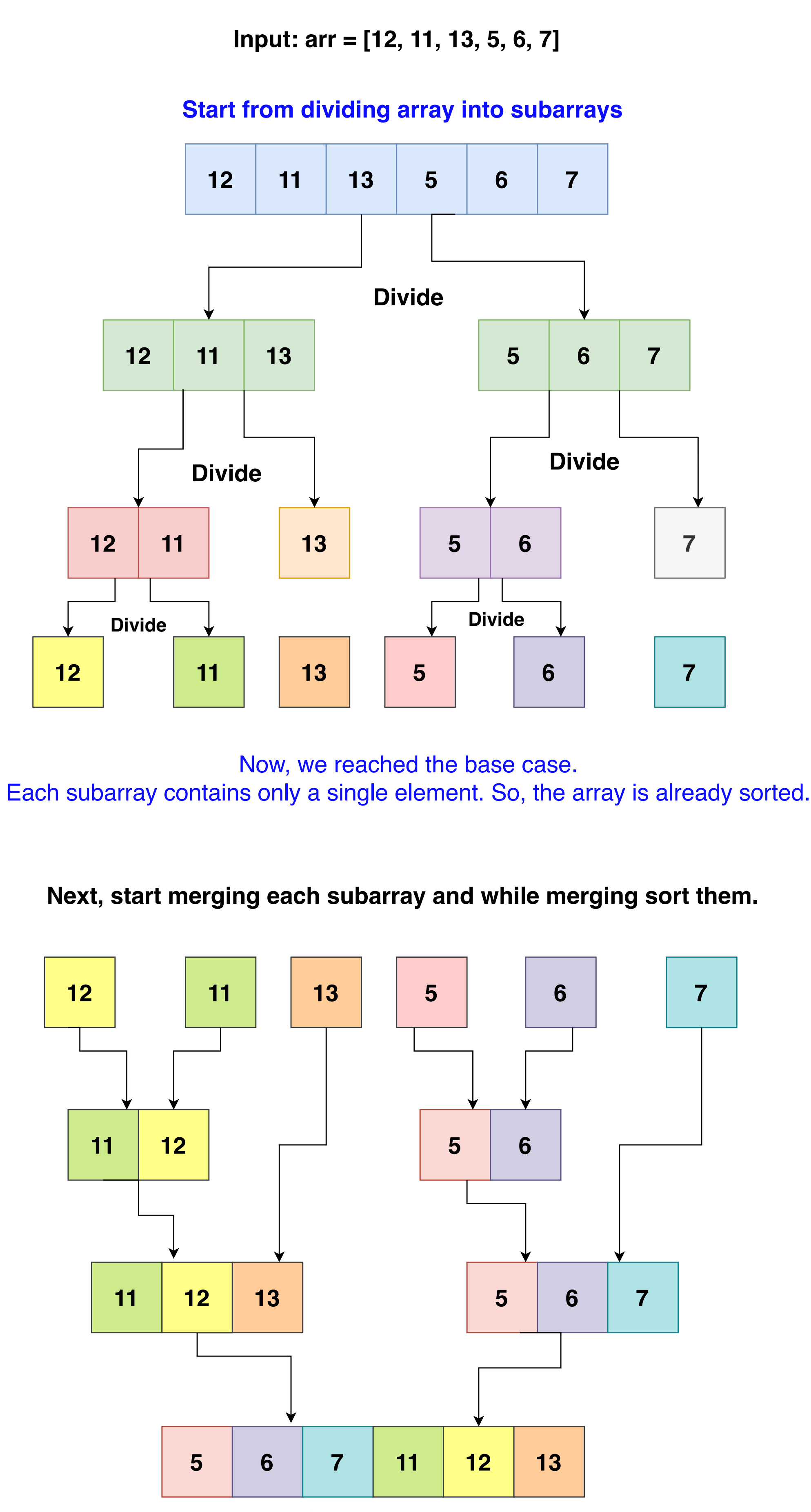
Algorithm

1. **MergeSort**(**arr, left, right**):
   - If `left < right`:
     - Find the middle point: `m = (left + right) / 2`.
     - Recursively sort the first half: `MergeSort(arr, left, m)`.
     - Recursively sort the second half: `MergeSort(arr, m + 1, right)`.
     - Merge the two sorted halves using `Merge(arr, left, m, right)`.
2. **Merge**(**arr, left, m, right**):
   - Determine the sizes of two subarrays to be merged: `sizeLeft = m - left + 1`, `sizeRight = right - m`.
   - Create temporary arrays `L[]` (for the left subarray) and `R[]` (for the right subarray).
   - Copy data to `L[]` from `arr[left...m]` and `R[]` from `arr[m+1...right]`.
   - Initialize three pointers: `i = 0` (for traversing L[]), `j = 0` (for traversing R[]), and `k = left` (for the merged array).
   - Merge arrays `L[]` and `R[]` back into `arr[left...right]`:
     - Compare each element of `L[]` with `R[]` and copy the smaller element into `arr[k]`.
     - Increment `i` or `j` based on which element was copied, and increment `k`.
   - Copy any remaining elements in `L[]` and `R[]` to `arr[]`, if there are any.

Key Points

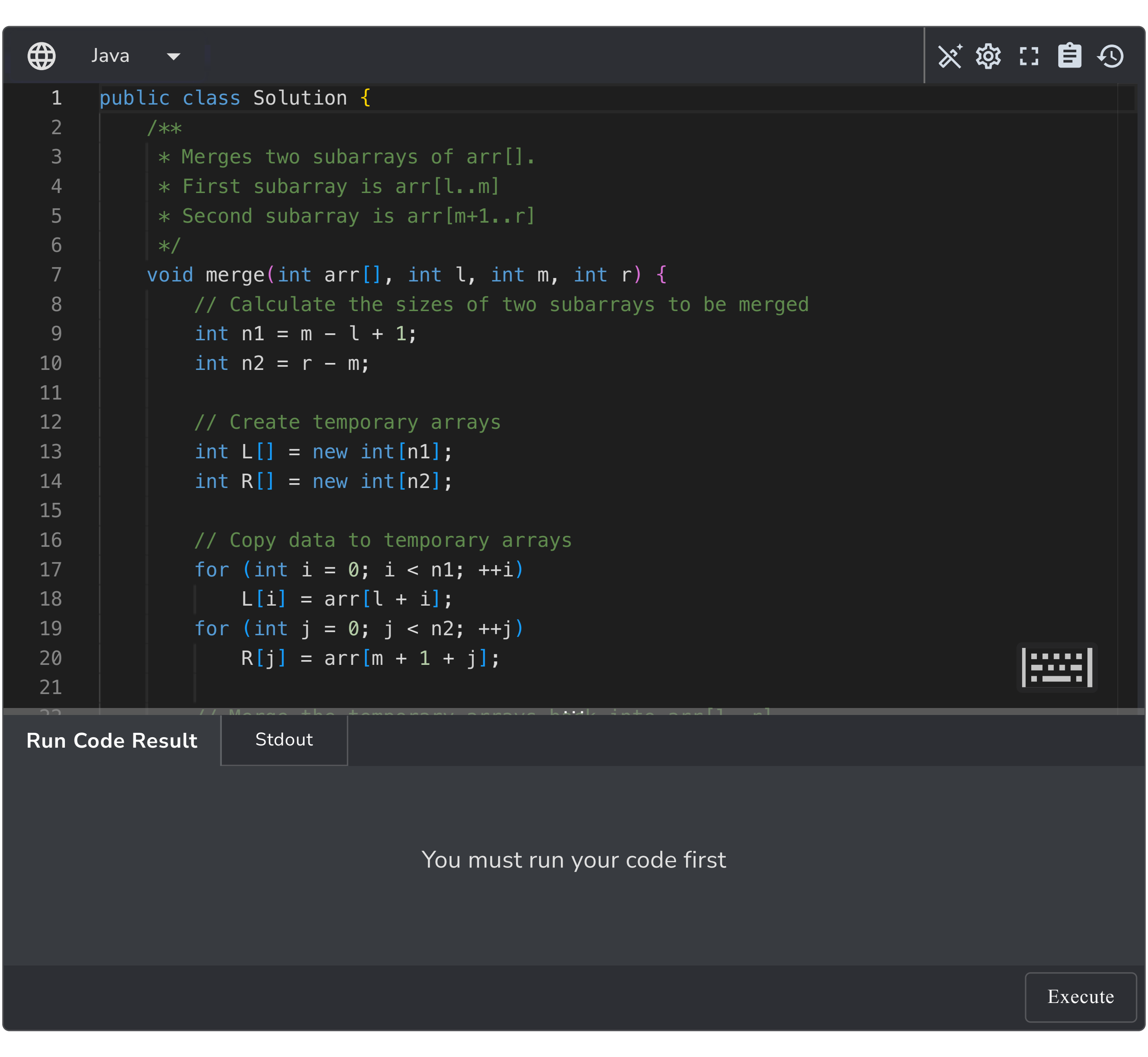
- **Divide:** The array is divided into two halves using the `MergeSort` function recursively.
- **Conquer:** Each half is sorted recursively using the same `MergeSort` function.
- **Combine:** The sorted halves are merged into a single sorted array using the `Merge` function.

Algorithm Walkthrough


Input: arr = [12, 11, 13, 5, 6, 7]
Start from dividing array into subarrays
12 11 13 5 6 7
Divide
12 11 13
5 6 7
Divide
Divide
12 11
13
5 6
7
Divide
Divide
12
11
13
5
6
7
Now, we reached the base case.
Each subarray contains only a single element. So, the array is already sorted.
Next, start merging each subarray and while merging sort them.
12
11
13
5
6
7
11 12
5 6
11 12 13
5 6 7
5 6 7 11 12 13

Sorted Array.

Code



```java
public class Solution {
    /**
     * Merges two subarrays of arr[].
     * First subarray is arr[l..m]
     * Second subarray is arr[m+1..r]
     */
    void merge(int arr[], int l, int m, int r) {
        // Calculate the sizes of two subarrays to be merged
        int n1 = m - l + 1;
        int n2 = r - m;

        // Create temporary arrays
        int L[] = new int[n1];
        int R[] = new int[n2];

        // Copy data to temporary arrays
        for (int i = 0; i < n1; ++i)
            L[i] = arr[l + i];
        for (int j = 0; j < n2; ++j)
            R[j] = arr[m + 1 + j];

```

You must run your code first

Time Complexity

The time complexity of Merge Sort can be represented by the recurrence relation:

```
T(n) = O(1)  if n is small (typically if  n < 1)

f_1(n) + 2T(n/2) + f_2(n) otherwise
```

Where:

- (T(n)) is the time complexity for sorting an array of size (n).
- (f_1(n)) represents the time taken to divide the array, which is (O(1)) since it's just calculating the middle index.
- 2T(n/2) represents the time taken for sorting the two halves of the array recursively.
- (f_2(n)) represents the time taken for merging the two halves, which is (O(n)), as it involves comparing and combining (n) elements in total.

Solving this recurrence relation, we get the overall time complexity of Merge Sort as (O(n log n)).

Space Complexity

The space complexity of Merge Sort is primarily due to the temporary arrays used for merging and the space used for recursive function calls. It can be summarized as follows:

- **Temporary Arrays:** During each merge, a temporary array of size (n) is used, leading to (O(n)) space complexity.

Therefore, the overall space complexity of Merge Sort is (O(n)).

Applications of Divide and Conquer Approach

- **Sorting Algorithms:** Used in Merge Sort and Quick Sort, efficiently sorting data.
- **Multiplying Large Numbers:** Utilized in algorithms like Karatsuba multiplication.
- **Matrix Multiplication:** Efficiently multiplying matrices using Strassen's algorithm.
- **Closest Pair of Points Problem:** Solving it quickly in computational geometry.
- **Fast Fourier Transform (FFT):** For polynomial multiplication and signal processing.
- **Solving Recurrence Relations:** Like the Master Theorem for algorithm analysis.
- **Tower of Hanoi Problem:** Classical problem solved optimally using this approach.

Advantages of Divide and Conquer

- **Efficient for Large Problems:** Breaks complex problems into manageable subproblems.
- **Parallel Processing:** Subproblems can often be solved in parallel, improving efficiency.
- **Algorithmic Efficiency:** Often leads to algorithms with good time complexity.
- **Simplicity:** Simplifies the algorithm design and implementation process.

Disadvantages of Divide and Conquer

- **Recursion Overhead:** Heavy use of recursion can lead to performance issues.
- **Stack Overflow:** Deep recursion can cause stack overflow in languages with limited stack size.
- **Memory Usage:** Can require additional memory for maintaining recursive calls and subproblems.
- **Complexity in Merge Step:** The combine or merge step can be complex to implement efficiently.
- **Not Always Optimal:** For certain problems, other techniques like dynamic programming might be more efficient, especially in cases with overlapping subproblems.

Divide and Conquer vs Dynamic Programming

Both Divide and Conquer (D&C) and Dynamic Programming (DP) are algorithmic paradigms that solve problems by breaking them down into smaller subproblems. However, they differ significantly in how they approach these subproblems:

1. Approach to Subproblems

- **Divide and Conquer:** It divides the problem into independent subproblems and solves each one separately. The solutions to these subproblems do not depend on each other.
- **Dynamic Programming:** DP also breaks problems down into subproblems, but these subproblems are not independent. DP solves each subproblem only once and stores the result in a table, avoiding the work of recalculating the answer every time the subproblem occurs.

2. Use Cases

- **Divide and Conquer:** Best for problems with unique subproblems like quicksort or merge sort.
- **Dynamic Programming:** Ideal for problems where the same subproblems are solved multiple times, such as computing the nth Fibonacci number or solving the Knapsack problem.

Example: Fibonacci Number Calculation

- **Divide and Conquer Approach** (Simple Recursion):

  ```python
  def fib(n):
      if n ≤ 1:
          return n
      return fib(n-1) + fib(n-2)
  ```

  This approach recalculates the same Fibonacci numbers many times.

- **Dynamic Programming Approach** (Memoization):

  ```python
  def fib(n, memo={}):
      if n ≤ 1:
          return n
      if n not in memo:
          memo[n] = fib(n-1, memo) + fib(n-2, memo)
      return memo[n]
  ```

  This approach stores the results of subproblems, preventing redundant calculations.

D&C works well for problems with distinct subproblems, while DP is more efficient for problems with overlapping subproblems due to its ability to store and reuse previous results.

Let's start solving the problem on Divide and conquer algorithm in the next chapters.

← Previous
Solution: Removing Minimum And Maximum From Array
Next →
Majority Element (Easy)
☐ Mark as Completed